

Software Requirements Specification (SRS)

Minor Project Management Portal

IIIT Naya Raipur

Prepared by:

Shreyash Rai

B.Tech in Data Science and Artificial Intelligence (DSAI)

Vinit Yadav

B.Tech in Electronics and Communication Engineering (ECE)

Under the Guidance of:

Dr. Vinay Kumar

Version: 1.0

Date: April 2026

Dr. SPM International Institute of Information Technology, Naya Raipur

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Intended Audience	2
1.4	Definitions and Abbreviations	2
2	Overall Description	2
2.1	Product Perspective	2
2.2	Product Functions (High-Level)	3
2.3	User Classes and Characteristics	3
2.4	Operating Environment	3
2.5	Assumptions and Dependencies	3
3	Functional Requirements	3
3.1	Authentication Module	3
3.2	Student Module	4
3.3	Faculty Module	4
3.4	Admin Module	5
3.5	Notification Module	5
4	Non-Functional Requirements	6
4.1	Security	6
4.2	Performance	6
4.3	Usability	6
4.4	Maintainability	6
5	System Architecture	6
6	Data Models (Key Entities)	7
7	External Interface Requirements	7
7.1	Hardware Interfaces	7
7.2	Software Interfaces	7
7.3	Communication Interfaces	7
8	Appendix A — Environment Variables	8

1 Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for the **Minor Project Management Portal**, a web-based system developed to digitize and streamline the administration of B.Tech. minor projects at IIIT Naya Raipur. It serves as the authoritative reference for developers, testers, and stakeholders.

1.2 Scope

The portal replaces manual, paper-based tracking of minor projects with an integrated digital workflow covering:

- Student group formation and faculty mentor assignment
- Project proposal submission, review, and approval
- Progress update tracking between students and mentors
- Mid-term and end-term panel evaluations with rubric-based mark entry
- Admin oversight, data import/export, and account management
- Automated email notifications at each workflow milestone

The system is accessible via web browser by three roles: **Students**, **Faculty**, and **Admin**.

1.3 Intended Audience

- Software development team
- Academic administrators and faculty coordinators
- Quality assurance testers
- Future maintainers

1.4 Definitions and Abbreviations

Term	Definition
Minor Project	A semester-long project undertaken by a student group in their 5th–8th semester
Group	2–3 students registered together for a shared minor project
Panel	A set of faculty members assigned to evaluate a batch of groups
Cohort / Batch	Students sharing the same year of admission (e.g., 2022-batch)
OTP	One-Time Password used for account activation
SRS	Software Requirements Specification
JWT	JSON Web Token used for session authentication
XLSX	Microsoft Excel Open XML spreadsheet format

2 Overall Description

2.1 Product Perspective

The portal is a standalone MERN-stack web application (MongoDB, Express.js, React, Node.js) with TypeScript on both the client and server. It communicates over a REST API and uses JWT-based stateless authentication. Email notifications are sent via SMTP (Nodemailer).

2.2 Product Functions (High-Level)

1. **Authentication** — Login, OTP-based account activation, JWT session management
2. **Group Formation** — Student-driven group creation with faculty mentor selection
3. **Project Proposals** — Group submission, faculty review (approve/reject with feedback)
4. **Progress Updates** — Students post updates; faculty monitors and gives per-student feedback
5. **Evaluations** — Mid-term and end-term rubric-based assessments by guide and panel
6. **Admin Management** — User directory, import/export, event lifecycle, panel assignment
7. **Archival** — Previous-semester groups/projects archived when a new semester starts
8. **Notifications** — Automated emails at key events

2.3 User Classes and Characteristics

Role	Description	Notable Capabilities
Student	Enrolled B.Tech. student	Form groups, submit proposals, post updates, view evaluations
Faculty	Teaching staff member	Approve proposals, mentor groups, enter evaluation marks, view panel groups
Admin	System administrator	Full access: manage users, events, panels, imports, exports

2.4 Operating Environment

- **Client:** Modern web browser (Chrome $\geq$ 110, Edge $\geq$ 110, Firefox $\geq$ 109)
- **Server:** Node.js $\geq$ 18 on Linux/macOS/Windows
- **Database:** MongoDB $\geq$ 6.0 (local or Atlas)
- **Network:** HTTPS in production; HTTP locally acceptable

2.5 Assumptions and Dependencies

- A valid SMTP server credential is available for email delivery.
- A valid JWT_SECRET environment variable is set before deployment.
- Administrators perform the one-time “Mark Students Inactive” migration when transitioning to a new academic semester.

3 Functional Requirements

3.1 Authentication Module

ID	Requirement
FR-AUTH-01	The system shall authenticate users with an email/password login form.
FR-AUTH-02	When a student account exists but <code>isActive = false</code> , the login response shall include <code>requiresActivation: true</code> and trigger an OTP email to the student.
FR-AUTH-03	Students shall enter the 6-digit OTP on the Login page to activate their account. Successful OTP entry shall set <code>isActive = true</code> and <code>isVerified = true</code> .
FR-AUTH-04	Authentication tokens shall be JWTs signed with the JWT_SECRET environment variable. The server shall reject any JWT signed with a different secret.
FR-AUTH-05	Tokens shall expire after 1 day. Expired tokens shall return HTTP 401.
FR-AUTH-06	The server shall validate the <code>x-auth-token</code> header on all protected routes.

3.2 Student Module

ID	Requirement
FR-STU-01	A student shall be able to view the student directory filtered to their own batch/cohort only.
FR-STU-02	A student with no group shall be able to create a new group by selecting 1–2 other students from the directory and a faculty mentor (max total size 3).
FR-STU-03	Group formation shall validate that all selected members are ungrouped, active, and belong to the same cohort (unless explicitly overridden for droppers).
FR-STU-04	A student who is a group member shall be able to submit a project proposal (title, description, tags, faculty assignment).
FR-STU-05	A student shall be able to post progress updates (text, links, file attachments) on any active project.
FR-STU-06	A student shall be able to view their mentor’s feedback and per-student comments.
FR-STU-07	A student shall be able to submit files (PDF, PPT, ZIP) during an active evaluation event.
FR-STU-08	A student shall be able to view all archived projects and groups from previous semesters.

3.3 Faculty Module

ID	Requirement
FR-FAC-01	A faculty member shall see the list of all groups assigned to them as mentor.
FR-FAC-02	A faculty member shall be able to approve or reject a pending project proposal, with optional textual feedback.
FR-FAC-03	A faculty member shall be able to post progress updates on projects they mentor.
FR-FAC-04	A faculty member shall be able to enter mid-term and end-term evaluation marks for each group they guide, in either rubric or direct entry mode.
FR-FAC-05	A faculty member who is assigned to a panel shall be able to enter panel evaluation marks for groups in their panel.
FR-FAC-06	A faculty member shall be able to leave per-student written feedback comments for each member of a group.
FR-FAC-07	Mark entry shall clamp inputs to the configured maximum per rubric field. Overflow shall be prevented both client-side (<code>max</code> attribute) and server-side (<code>Math.min</code> clamping).

3.4 Admin Module

ID	Requirement
FR-ADM-01	An admin shall be able to create, update, activate/deactivate, and delete academic events (Group Formation, Mid-Term, End-Term).
FR-ADM-02	The system shall prevent creation of a Mid-Term or End-Term event if a Group Formation event is still active (server-side guard).
FR-ADM-03	When a new Group Formation event is created, all existing non-archived groups and projects shall be automatically archived and mentors detached.
FR-ADM-04	An admin shall be able to create evaluation panels by assigning faculty members and a batch year.
FR-ADM-05	An admin shall be able to view and manage the full student directory with filtering by batch, branch, group status, and verification status.
FR-ADM-06	An admin shall be able to import students from an XLSX file (two-phase preview + commit), with per-row error reporting on duplicates and missing fields.
FR-ADM-07	An admin shall be able to export students, panels, and evaluations to formatted XLSX files.
FR-ADM-08	An admin shall be able to export and import a full database snapshot (JSON) for backup and restoration.
FR-ADM-09	An admin shall be able to execute a one-time “Mark Students Inactive” operation resetting all students for a new semester.
FR-ADM-10	The admin dashboard overview shall display: Total Students, Ungrouped Students, Activated/Unactivated Accounts, Total Groups, Total Faculty, Total Projects, and a group-status breakdown (Forming, ProposalPending, Approved, Dissolved).

3.5 Notification Module

ID	Requirement
FR-EMAIL-01	On successful group formation, the system shall send a confirmation email to all group members.
FR-EMAIL-02	On project proposal submission, the system shall send a notification email to the assigned faculty mentor.
FR-EMAIL-03	On proposal approval or rejection, the system shall send a status email to all group members including any written feedback.
FR-EMAIL-04	On evaluation panel creation, the system shall send a notification email to all assigned panel faculty.
FR-EMAIL-05	On event creation, the system shall send a notification email to all active students.
FR-EMAIL-06	When a student posts a project update, the system shall send a notification email to the assigned faculty mentor.
FR-EMAIL-07	Email delivery failures shall be logged as errors but shall not interrupt or fail the primary API request.

4 Non-Functional Requirements

4.1 Security

ID	Requirement
NFR-SEC-01	All passwords shall be stored as bcrypt hashes (salt rounds ≥ 10).
NFR-SEC-02	The <code>JWT_SECRET</code> environment variable shall be required at server startup; the server shall exit unconditionally with a fatal error if absent.
NFR-SEC-03	The <code>PUT /api/users/:id</code> route shall be protected by <code>adminAuth</code> middleware, allowing only admins to update user fields, including <code>role</code> .
NFR-SEC-04	Admin-only routes (<code>/api/admin/*</code> , import routes) shall require the <code>adminAuth</code> middleware.
NFR-SEC-05	Role-based filtering shall be applied on <code>GET /api/projects</code> : students see only their own group's project; faculty see only mentored projects; admins see all.

4.2 Performance

ID	Requirement
NFR-PERF-01	API responses for lists shall complete within 2 seconds on the local network under normal load.
NFR-PERF-02	Large student/group/project lists accessed by admins shall support <code>page</code> and <code>limit</code> query parameters for server-side pagination (max 200 per page).

4.3 Usability

ID	Requirement
NFR-UX-01	All modals shall be dismissible without page reload.
NFR-UX-02	Import operations shall show a preview step before committing changes to the database.
NFR-UX-03	All error messages shall be displayed inline in the UI; <code>window.alert()</code> shall not be the primary error mechanism for user-facing flows.

4.4 Maintainability

ID	Requirement
NFR-MAINT-01	No production deployment shall include <code>console.log</code> debug statements in server controllers or middleware.
NFR-MAINT-02	Server startup must log a 'MongoDB connected' message upon successful database connection.
NFR-MAINT-03	All API endpoints shall use the <code>/api/</code> prefix. The shared <code>api</code> Axios instance must be used by all client components.

5 System Architecture

React Client (Vite + TS)
Pages: Dashboard, AdminDashboard, MenteeGroup,
Login, GroupFormation (→ redirect)
Shared API util (Axios + JWT interceptor)

REST (+JWT)

Express.js Server (Node + TS)

Routes: auth, users, groups, projects,
events, panels, admin

Middleware: auth, adminAuth, multer (uploads)

MongoDB (ODM) SMTP (Nodemailer)
Mongoose 9.2.1 sendEmail utils

6 Data Models (Key Entities)

Model	Key Fields
User	name, email, password (hashed), role (Student/Faculty/Admin), rollNumber, branch, semester, department, isVerified, isActive, maxGroups, batchConfigs
Group	name, members[], status (Forming/ProposalPending/Approved/Dissolved), targetBatch, project, isArchived
Project	title, description, status (Draft/Pending/Approved/Rejected), group, faculty, updates[], midTermEvaluation, endTermEvaluation, submissions[], studentFeedback[], isArchived
Event	type (group_formation_project_proposal/mid_term_evaluation/end_term_evaluation), startDate, endDate, extensionDate, isActive, batchYear, rubricParams
Panel	faculty[], batchYear, room

7 External Interface Requirements

7.1 Hardware Interfaces

- Standard server hardware with internet access for production deployment
- No specialized hardware required

7.2 Software Interfaces

- MongoDB ≥ 6.0 database instance
- SMTP relay (Gmail, Zoho, institutional SMTP)

7.3 Communication Interfaces

- HTTP/HTTPS REST API
- SMTP for transactional email
- Browser localStorage for JWT persistence

8 Appendix A — Environment Variables

Variable	Required	Description
MONGO_URI	Yes	Full MongoDB connection string
JWT_SECRET	Yes	Secret for signing JWTs (min 32 chars recommended)
EMAIL_HOST	No	SMTP host (e.g., <code>smtp.gmail.com</code>)
EMAIL_PORT	No	SMTP port (e.g., <code>587</code>)
EMAIL_USER	No	SMTP username/email address
EMAIL_PASS	No	SMTP password or app password
EMAIL_SECURE	No	<code>true</code> for port 465, <code>false</code> otherwise
PORT	No	Server port (default: <code>5000</code>)
CLIENT_URL	No	Frontend origin for CORS (default: <code>http://localhost:5173</code>)